

**Détection automatisée des vulnérabilités
des smart contracts Solidity :
une approche basée sur
l'apprentissage profond**

par

Exauce KOMPANI KIPANGU

Octobre 2026

Résumé

Ce mémoire étudie la détection automatisée des vulnérabilités des smart contracts Solidity par apprentissage profond. SMART BUG associe un réseau convolutif et un réseau récurrent bidirectionnel pour analyser le code complet. Le corpus nettoyé comprend 18 238 contrats ou représentations de contrats, répartis par groupes entre apprentissage, validation, calibration, test et source réservée. Le CNN-BiLSTM actif, entraîné depuis zéro, obtient un F1 macro de 82,10 % et un rappel positif de 88,95 % sur le test. Sur les mêmes contrats, XGBoost atteint un F1 macro de 89,41 %. CodeBERT a été étudié comme candidat supplémentaire, sans entraînement complet ni score. La comparaison reste incomplète : le troisième modèle définitif n'est pas encore choisi. Ces résultats soutiennent une aide au tri des contrats à auditer, sous réserve des annotations et d'une validation indépendante.

Mots clés : smart contracts, Solidity, vulnérabilités, apprentissage profond, CNN-BiLSTM, calibration.

Remerciements

Personne n'écrit jamais un travail scientifique « tout seul », et dans mon cas c'est encore plus vrai que d'habitude. Ce travail de mémoire existe, non seulement grâce à mes idées personnelles, mais aussi parce que beaucoup d'autres personnes y ont concentré leur temps, leurs talents et leurs idées.

C'est aussi le résultat des cours auxquels nous avons assisté durant tous ces temps à la Faculté de Science et Technologie. C'est également le résultat de nos recherches dans une multitude de livres écrits par des professionnels.

Comme le dit un adage : « seul on va plus vite, ensemble on va plus loin ». Si tel est le cas, nous avons accumulé beaucoup d'expériences pendant l'élaboration de ce travail de mémoire de licence en droit. Toutefois, avant de mettre un point final à ce travail et de réfléchir à l'art de mener à bien un travail de mémoire, nous voudrions remercier certaines personnes pour leur soutien à la réalisation de ce travail.

En particulier, nous devons beaucoup de gratitude à Monsieur le Professeur Jordan Félicien MASAKUNA, directeur de ce présent travail. Celui qui nous a offert l'opportunité d'élaborer ce travail en nous faisant profiter de ses observations critiques et de son avis professionnel. Qu'il soit sincèrement remercié.

À ma mère Marquise MANKOBA qui m'a soutenu tout au long de mon cursus universitaire et sans qui je ne serais pas où je suis.

À mon tuteur Dode ESIMBO qui a toujours été là pour moi, ainsi qu'à mes oncles et tantes.

À Candide MAGOLU pour tes encouragements et m'avoir poussé chaque fois à me surpasser. Que nos frères et sœurs de la famille Bryan, Jérémie et Grace trouvent ici à travers ces lignes, l'expression d'un sentiment de joie familiale.

Notre gratitude s'adresse aussi à nos amis et connaissances pour leur soutien à la réalisation de ce travail : Bendi ESIMBO, Jean-Pierre SHUKRANI, Placide WANGI, Brummel DUASENGE, Jonathan-Bob TSHINGANI, Christian-Noah, Bourgeois KIMBONDO, Don MAKILA, TP groupe, Jean-Luc BANZA et tant d'autres.

Qu'il nous soit permis de conclure par une pensée particulière pour nos familles ; car personne ne peut évaluer la gratitude qui est due aux membres de la famille d'un autre. La compréhension et le flegme dont elles ont fait preuve pendant plusieurs années nous ont permis de mener à bien un travail qu'elles ont à juste titre trouvé trop exclusif.

Il est impossible d'énumérer tous à travers ces pages. Toute autre personne qui mérite notre reconnaissance, se sente vraiment remerciée, et trouve ici l'expression de notre profonde gratitude.

Dédicace

Je dédie ce travail à ma chère grand mère Angélique, dont le sourire illumine chaque instant

Table des matières

Résumé.....	i
Remerciements.....	ii
Dédicace.....	iv
Table des matières.....	v
Table des figures.....	vii
Liste des tableaux.....	viii
Liste des abréviations.....	ix
Introduction.....	1
1 Concepts théoriques de base.....	3
1.1 Notions de base.....	4
1.1.1 Blockchain, smart contracts et Solidity.....	4
1.1.2 Vulnérabilités et portée des annotations.....	5
1.1.3 Approches de détection.....	6
1.1.4 Représentations profondes du code.....	7
1.1.5 Apprentissage, validation et calibration.....	8
1.1.6 Critères d'évaluation.....	9
1.2 Revue de littérature.....	10
1.2.1 Positionnement de notre travail.....	11
2 Architecture et modélisation.....	12
2.1 Architecture du système SMART BUG.....	13
2.1.1 Construction et séparation du corpus.....	14

2.1.2 Prétraitement et prévention des fuites.....	15
2.1.3 Réseau hiérarchique retenu.....	16
2.2 Modélisation en UML.....	17
2.2.1 Diagramme de classes.....	17
2.2.2 Diagramme de cas d'utilisation.....	18
2.2.3 Diagramme de séquence.....	19
2.2.4 Diagramme d'états-transitions.....	19
2.2.5 Organisation des fichiers et reproductibilité.....	20
3 Investigation expérimentale.....	21
3.1 Description de l'application.....	21
3.1.1 Technologies utilisées et hyperparamètres.....	22
3.1.2 Mise en œuvre de l'entraînement.....	23
3.1.3 Intégration et cohérence de l'inférence.....	24
3.2 Résultats.....	25
3.2.1 Performances du modèle actif.....	25
3.2.2 Probabilités et calibration.....	26
3.2.3 Comparaison du CNN-BiLSTM, de XGBoost et de CodeBERT.....	27
3.2.4 Protocole commun d'apprentissage et d'évaluation.....	28
3.2.5 Fine-tuning, calibration et coût de calcul.....	29
3.3 Discussion.....	30
3.4 Potentielles améliorations.....	31
Conclusion.....	32
Bibliographie.....	33

Table des figures

1.1 Place de l'analyse dans le cycle d'un contrat.....	4
1.2 Principales approches et rôle commun de l'évaluation.....	6
2.1 Chaîne de données, apprentissage et utilisation.....	13
2.2 Architecture du modèle actif CNN-BiLSTM.....	16
2.3 Modèle conceptuel des données et du prédicteur.....	17
2.4 Cas d'utilisation de l'application SMART BUG.....	18
2.5 Interactions principales lors d'une analyse.....	19
2.6 États simplifiés du traitement d'un contrat.....	19
3.1 Pertes d'apprentissage et de validation, graine 73.....	23
3.2 Exactitudes d'apprentissage et de validation, graine 73.....	24
3.3 Matrice de confusion du modèle actif calibré.....	25
3.4 Fiabilité des probabilités sur le test, dix intervalles.....	26
3.5 Deux modèles évalués et un candidat supplémentaire.....	27
3.6 Rôle des cinq partitions dans la comparaison.....	28
3.7 Agrégation par contrat et calibration des décisions.....	29

Liste des tableaux

1.1 Familles de vulnérabilités considérées.....	5
1.2 Définition des indicateurs issus de la confusion.....	9
1.3 Démarches des travaux connexes.....	10
2.1 Composition du corpus final par partition.....	14
2.2 Principaux fichiers et responsabilités.....	20
3.1 Réglages du CNN-BiLSTM actif.....	22
3.2 Résultats du test pour le CNN-BiLSTM actif.....	26
3.3 État de la comparaison à trois modèles.....	27

Liste des abréviations

AdamW : Adam avec décroissance des poids découplée
API : Application Programming Interface : interface de programmation
AUC : Area Under the Curve : aire sous la courbe
BERT : Bidirectional Encoder Representations from Transformers
BiLSTM : Bidirectional Long Short-Term Memory : LSTM bidirectionnel
BPE : Byte Pair Encoding : encodage en sous-tokens
CBGRU : Nom du modèle hybride de Zhang et al. (2022)
CGT : Consolidated Ground Truth : vérité terrain consolidée
CLS : Classification token : marqueur de représentation de fenêtre
CNN : Convolutional Neural Network : réseau neuronal convolutif
CodeBERT : Encodeur préentraîné pour le code et le langage naturel
CPU : Central Processing Unit : processeur central
CSS : Cascading Style Sheets : feuilles de style en cascade
DOI : Digital Object Identifier : identifiant pérenne d’une publication
ECE : Expected Calibration Error : erreur de calibration estimée
EMNLP : Empirical Methods in Natural Language Processing
EVM : Ethereum Virtual Machine : machine virtuelle Ethereum
F1 : Moyenne harmonique de la précision et du rappel
FN : Faux négatif
FP : Faux positif
FPR : False Positive Rate : taux de faux positifs
HTML : HyperText Markup Language : langage de balisage hypertexte
IA : Intelligence artificielle
IC : Intervalle de confiance
IJCAI : International Joint Conference on Artificial Intelligence
JSON : JavaScript Object Notation : format de données structurées
KDD : Knowledge Discovery and Data Mining : conférence sur l’analyse des données
L2 : Régularisation quadratique
LSTM : Long Short-Term Memory : mémoire récurrente à long et court terme
MIT : Massachusetts Institute of Technology
PMLR : Proceedings of Machine Learning Research
PR-AUC : Precision–Recall Area Under the Curve : aire précision-rappel
ReLU : Rectified Linear Unit : unité linéaire rectifiée
ROC : Receiver Operating Characteristic : courbe rappel / faux positifs
s. d. : Sans date de publication précisée
SHA-256 : Secure Hash Algorithm, empreinte cryptographique de 256 bits
SQL : Structured Query Language : langage de requêtes structurées
TF-IDF : Term Frequency–Inverse Document Frequency : pondération lexicale
UML : Unified Modeling Language : langage de modélisation unifié
UTF-8 : Unicode Transformation Format, encodage en unités de 8 bits
VN : Vrai négatif
VP : Vrai positif
WETSEB : Workshop on Emerging Trends in Software Engineering for Blockchain
XGBoost : eXtreme Gradient Boosting : boosting d’arbres

Introduction

Les smart contracts rendent possible l'exécution de règles programmées sur une blockchain. Leur utilisation engage toutefois la fiabilité du code : une instruction erronée, un contrôle d'accès insuffisant ou une interaction mal maîtrisée peut compromettre le comportement attendu. La transparence du registre ne garantit pas la sécurité des programmes qui y sont déployés. L'examen du code Solidity constitue donc une étape importante de leur développement.

L'audit manuel permet une analyse contextualisée, mais exige du temps et une expertise spécialisée. Les outils automatiques facilitent le repérage des anomalies ; leurs alertes doivent néanmoins être interprétées. L'apprentissage profond propose une autre démarche : apprendre, à partir de contrats annotés, des représentations associées à la présence de vulnérabilités. Sa pertinence dépend autant des données et du protocole expérimental que de la complexité du réseau.

La problématique de ce mémoire est la suivante : dans quelle mesure un modèle profond analysant le code Solidity complet peut-il aider à détecter des contrats potentiellement vulnérables, tout en maîtrisant les fuites de données, les faux positifs et l'interprétation de ses probabilités ? La question porte sur une aide à l'audit ; elle ne suppose pas qu'un score statistique puisse certifier la sécurité d'un contrat.

L'objectif général consiste à concevoir et évaluer SMART BUG, un système de classification binaire associé à une application d'analyse. Les objectifs spécifiques sont de constituer un corpus traçable, de limiter les recouvrements entre partitions, de traiter les contrats sans supprimer leur fin et de calibrer les scores sur des données dédiées. La comparaison prévue oppose le CNN-BiLSTM actif à XGBoost et à CodeBERT sur les mêmes contrats.

Trois hypothèses guident la comparaison. Le CNN-BiLSTM pourrait apprendre des régularités locales et séquentielles utiles à la détection. XGBoost pourrait fournir une référence classique compétitive à partir des fréquences lexicales. CodeBERT, adapté par fine-tuning, pourrait bénéficier de représentations préentraînées. Ces hypothèses demandent des mesures communes ; la supériorité d'une méthode n'est pas présumée.

La démarche associe une revue de littérature ciblée, la préparation des données et une évaluation quantitative. L'entraînement apprend les poids ; la validation sélectionne les états ; la calibration règle les probabilités et les seuils. Le test et une source réservée servent à l'évaluation. Le CNN-BiLSTM et XGBoost sont évalués ; les résultats de CodeBERT restent à compléter. La comparaison reste exploratoire, car le test avait déjà été consulté pendant le développement.

Le périmètre est celui du code source Solidity et d'un verdict global par contrat. Le réseau ne localise pas directement une ligne fautive et ne prédit pas une catégorie de vulnérabilité. Les alertes détaillées de l'application proviennent d'un moteur statique distinct. Cette séparation évite de présenter une règle experte comme une explication produite par le réseau.

Le mémoire comprend trois chapitres. Le premier présente les concepts et les travaux connexes. Le deuxième décrit l'architecture, les données et la modélisation. Le troisième expose l'implémentation, les résultats, leur discussion et les améliorations possibles. La conclusion répond à la problématique à partir des observations effectivement obtenues.

Chapitre 1

Concepts théoriques de base

La détection des vulnérabilités des smart contracts se situe à l'intersection de la sécurité logicielle, de la blockchain et de l'apprentissage automatique. Elle exige de distinguer le fonctionnement d'un programme, les annotations disponibles et la décision rendue par un outil. Un contrat classé positivement présente un signal compatible avec les vulnérabilités apprises ; cette classification ne démontre pas, à elle seule, l'existence d'un scénario d'exploitation.

Ce chapitre expose les notions nécessaires à la compréhension du système. Il introduit Solidity, les principales familles de faiblesses retenues et les représentations utilisées par les réseaux. Il présente ensuite les critères d'évaluation ainsi que des travaux proches du sujet. Les références sont choisies pour éclairer les décisions de conception ; leurs scores ne sont pas directement comparés aux nôtres lorsque les corpus et les protocoles diffèrent.

L'enjeu central est de relier chaque affirmation à son niveau de preuve. Une performance sur un corpus annoté décrit une capacité de reproduction de ces annotations. Une utilité opérationnelle exige, en complément, des données représentatives, une maîtrise des alertes inutiles et une vérification humaine des cas importants.

1.1 Notions de base

1.1.1 Blockchain, smart contracts et Solidity

Une blockchain conserve un historique partagé de transactions selon des règles communes de validation. Un smart contract est un programme qui met à jour un état lorsqu'une transaction ou un appel l'exécute. Sur Ethereum, Solidity permet de définir cet état, des fonctions et des conditions d'accès ; le code compilé s'exécute dans l'Ethereum Virtual Machine, ou EVM.

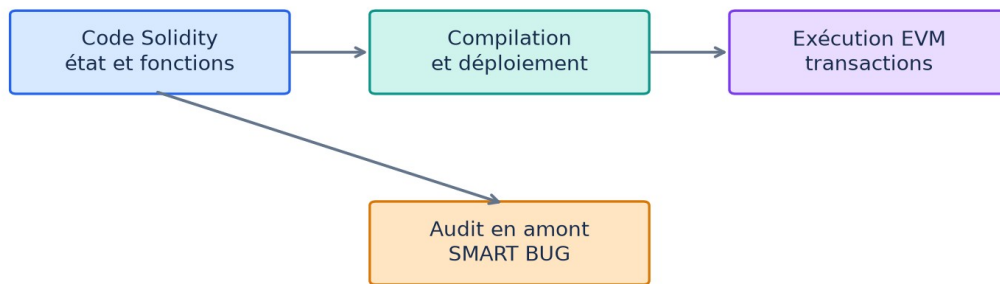


Figure 1.1 – Place de l'analyse dans le cycle d'un contrat.

Source : élaboration personnelle.

La figure 1.1 situe SMART BUG avant le déploiement, au stade où le code peut encore être corrigé. Le système accepte aussi l'examen de sources déjà publiées, mais n'exécute pas une transaction sur la blockchain pour produire son verdict. Il analyse une représentation du programme fourni.

L'exécution déterministe d'un contrat ne signifie pas que son comportement répond correctement à l'intention du développeur. Les interactions avec d'autres contrats, l'ordre des transactions et les limites de ressources constituent des éléments de sécurité à examiner. La documentation Solidity recommande notamment de considérer les appels externes et les limites de gaz dans la conception des contrats (SOLIDITY, s. d.).

1.1.2 Vulnérabilités et portée des annotations

Le corpus regroupe des annotations de provenances différentes. Sept familles servent notamment à définir le périmètre de contrôle de la source CGT. Elles ne constituent pas sept sorties du réseau : SMART BUG apprend une cible binaire indiquant la présence ou l'absence d'une vulnérabilité annotée.

Tableau 1.1 – Familles de vulnérabilités considérées.

Famille	Principe du risque
Arithmetic	Erreurs liées aux opérations et limites numériques
Unchecked low calls	Résultat d'un appel de bas niveau non vérifié
Reentrancy	Réentrée pendant une interaction externe
Denial of service	Blocage ou indisponibilité d'une fonction
Front running	Exploitation de l'ordre des transactions
Time manipulation	Dépendance fragile à l'horodatage
Bad randomness	Aléa prévisible ou influençable

Source : élaboration personnelle à partir des références citées.

Le sens d'un label négatif dépend de la couverture de l'évaluation d'origine. Si une source ne vérifie qu'une faiblesse particulière, l'absence de cette faiblesse ne prouve pas l'absence de toutes les autres. Le projet exclut donc les négatifs CGT dont le périmètre évalué ne couvre pas les sept familles retenues.

Les versions du langage et les mécanismes de protection influencent également l'interprétation du code. Une opération arithmétique ou un appel externe ne suffit pas à conclure à une faille. La documentation officielle décrit des précautions de conception, mais l'application de ces précautions doit être analysée dans le contexte du contrat (SOLIDITY, s. d.).

1.1.3 Approches de détection

L'analyse statique examine un programme sans l'exécuter. Elle peut s'appuyer sur des règles lexicales, des représentations intermédiaires ou des relations entre instructions. Slither illustre une approche structurée : sa représentation SlithIR facilite notamment les analyses de flux de données (FEIST, GRIECO et GROCE, 2019). Les règles du moteur local de SMART BUG sont plus simples ; elles ne doivent pas être assimilées à l'intégralité de Slither.

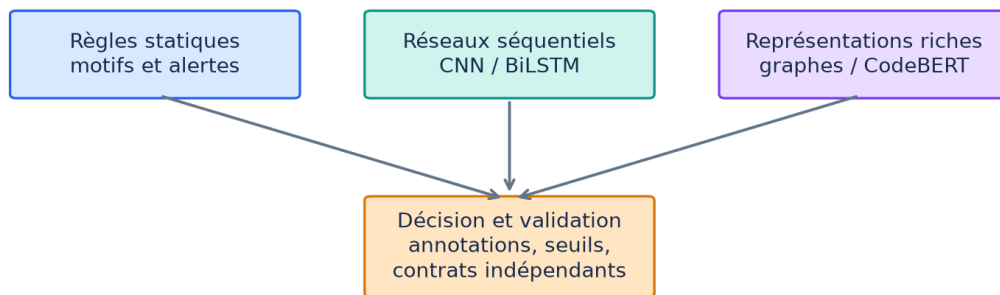


Figure 1.2 – Principales approches et rôle commun de l'évaluation.

Source : élaboration personnelle.

L'apprentissage supervisé utilise des couples constitués d'une entrée et d'un label pour ajuster un classificateur. Les représentations profondes sont apprises au cours de l'optimisation, tandis qu'une référence classique peut utiliser des fréquences de tokens. Les réseaux séquentiels, les graphes et les modèles préentraînés offrent différentes manières de représenter le code.

Ces approches présentent des compromis : simplicité d'intégration, richesse du contexte, coût de calcul et dépendance aux annotations. Dans ce travail, le classificateur et le moteur statique sont complémentaires dans le rapport affiché. Leurs scores restent séparés, afin de ne pas construire une probabilité combinée sans validation statistique.

1.1.4 Représentations profondes du code

Un token est une unité issue de l'analyse lexicale du code : mot réservé, opérateur, identifiant ou littéral. Une couche de plongement, ou *embedding*, associe à chaque token un vecteur numérique appris. Le réseau peut ainsi rechercher des régularités dans les séquences sans imposer manuellement une liste exhaustive de motifs vulnérables. L'apprentissage de représentations constitue un principe central de l'apprentissage profond (GOODFELLOW, BENGIO et COURVILLE, 2016).

Un réseau convolutif, ou CNN, applique des filtres sur des voisinages de tokens. Il produit des activations sensibles à certains motifs locaux. Une agrégation par moyenne et maximum résume ces activations en un vecteur de taille fixe. L'usage de convolutions sur des séquences a notamment été étudié pour la classification de phrases (KIM, 2014) ; son application au code nécessite toutefois une évaluation propre au domaine.

Un réseau LSTM maintient un état récurrent régulé par des portes. Sa variante bidirectionnelle, le BiLSTM, parcourt la séquence dans les deux sens. Dans SMART BUG, la récurrence porte sur les vecteurs des fenêtres du contrat. Cette organisation hiérarchique réduit la longueur de la séquence récurrente par rapport à une récurrence appliquée directement à tous les tokens.

L'association CNN-BiLSTM vise donc à articuler motifs locaux et relations entre portions du contrat. Elle ne construit ni arbre syntaxique ni graphe explicite de dépendances. Le code complet est fourni sous une forme normalisée, mais la compression des représentations peut perdre des informations utiles. La conservation de l'entrée et la compréhension sémantique exhaustive sont deux propriétés différentes.

Les poids du modèle actif sont initialisés puis appris sur le corpus du projet. Un modèle préentraîné commence avec des paramètres issus d'une tâche antérieure. Le *fine-tuning* modifie ces paramètres pour la nouvelle tâche ; il est prévu ici pour l'encodeur et la tête de CodeBERT. Entraîner seulement une tête sur un encodeur figé serait une autre stratégie de transfert et ne serait pas présenté comme un *fine-tuning* complet.

1.1.5 Apprentissage, validation et calibration

L'entraînement minimise une perte qui compare les sorties du réseau aux labels. Pour une classification binaire, le logit z est transformé en probabilité par la fonction sigmoïde. La perte binaire pénalise les prédictions éloignées de l'annotation. Les pondérations calculées sur l'entraînement ajustent la contribution des couples source/label ; elles ne corrigent pas automatiquement une annotation erronée.

La validation évalue les paramètres au fil des époques sans les apprendre directement. L'arrêt anticipé conserve le meilleur état selon la perte de validation lorsque les époques supplémentaires n'apportent plus d'amélioration. Le choix d'un modèle doit rester indépendant du test. Choisir après coup l'architecture présentant le meilleur score sur le test introduirait un optimisme dans l'estimation finale.

La calibration répond à une question distincte : les probabilités annoncées correspondent-elles aux fréquences observées ? Une température positive T ajuste l'échelle du logit, selon $p = 1 / (1 + \exp(-z/T))$. Cette transformation peut améliorer la calibration sans modifier l'ordre des scores. La méthode s'inscrit dans les approches étudiées par GUO et al. (2017).

Le seuil τ transforme la probabilité en décision : le label prédit vaut 1 lorsque $p \geq \tau$. Un seuil plus faible tend à détecter davantage de contrats positifs, au prix d'alertes supplémentaires sur les négatifs. Le seuil 0,5 n'est donc pas nécessairement adapté à un objectif de rappel élevé.

Dans ce travail, une partition distincte sert à ajuster la température et le seuil. Le rappel cible de 90 % est une contrainte vérifiée sur cette partition ; il ne constitue pas une garantie sur le test ni sur de futurs contrats. La calibration doit être réexaminée si la population analysée change, car la fréquence des vulnérabilités et les caractéristiques du code peuvent évoluer.

1.1.6 Critères d'évaluation

La matrice de confusion distingue les vrais positifs (VP), faux positifs (FP), vrais négatifs (VN) et faux négatifs (FN). Les positifs correspondent ici aux contrats annotés comme vulnérables. Ces quatre effectifs permettent de comprendre les erreurs concrètes d'un classificateur et de calculer les métriques suivantes.

Tableau 1.2 – Définition des indicateurs issus de la confusion.

Indicateur	Expression
Exactitude	$(VP + VN) / (VP + VN + FP + FN)$
Précision positive	$VP / (VP + FP)$
Rappel positif	$VP / (VP + FN)$
F1 positif	$2 \times \text{précision} \times \text{rappel} / (\text{précision} + \text{rappel})$
FPR	$FP / (FP + VN)$

Source : élaboration personnelle à partir des références citées.

Le F1 macro est la moyenne du F1 calculé pour chacune des deux classes. Il donne le même poids aux classes, même lorsque leurs effectifs diffèrent. La ROC AUC mesure la capacité de classement sur l'ensemble des seuils ; elle ne décrit pas à elle seule la qualité du seuil utilisé dans l'application.

Le score de Brier est la moyenne des erreurs quadratiques entre probabilités et labels. L'ECE estime l'écart entre probabilités moyennes et fréquences positives dans des intervalles de scores. Le projet utilise dix intervalles. Ces mesures complètent la discrimination ; une bonne séparation des classes peut coexister avec des probabilités mal calibrées.

Les moyennes sur trois graines décrivent la variabilité des entraînements exécutés. Les intervalles bootstrap par groupes estiment une autre composante d'incertitude, liée aux contrats évalués. Ils ne doivent pas être interprétés comme une couverture de tous les changements possibles de données, de matériel ou d'hyperparamètres.

1.2 Revue de littérature

Les travaux retenus illustrent plusieurs familles de solutions. TANN et al. (2018) étudient l'apprentissage séquentiel de faiblesses de smart contracts avec des LSTM. ZHUANG et al. (2020) proposent une représentation par graphe pour exploiter les relations du programme. ZHANG et al. (2022) présentent CBGRU, qui combine des représentations de mots et des méthodes profondes dans un modèle hybride.

Tableau 1.3 – Démarches des travaux connexes.

Travail	Représentation / démarche
Tann et al. (2018)	Apprentissage séquentiel LSTM
Zhuang et al. (2020)	Réseau neuronal sur graphe
Zhang et al. (2022)	CBGRU, modèle hybride
Feng et al. (2020)	CodeBERT, encodeur préentraîné
Chen et Guestrin (2016)	XGBoost, arbres par boosting
Ferreira et al. (2020)	Framework SmartBugs

Source : élaboration personnelle à partir des références citées.

CodeBERT est un encodeur préentraîné sur du code et du langage naturel (FENG et al., 2020). Les six langages de son préentraînement sont Python, Java, JavaScript, PHP, Ruby et Go ; Solidity n'en fait pas partie. Son adaptation doit donc être vérifiée sur notre corpus. XGBoost apprend un ensemble d'arbres par boosting (CHEN et GUESTRIN, 2016) ; il fournit une référence d'apprentissage classique.

SmartBugs fournit un cadre pour exécuter et comparer des outils d'analyse de contrats (FERREIRA et al., 2020). Il faut distinguer ce projet externe de SMART BUG, nom de l'application développée dans ce mémoire. Le corpus du présent travail conserve certaines provenances liées à SmartBugs ; notre réseau n'est pas une version de son framework.

1.2.1 Positionnement de notre travail

Cette étude ne revendique pas l’invention des CNN, des BiLSTM ou de leur association. Elle examine leur utilisation dans une chaîne complète de préparation, d’apprentissage, de calibration et de restitution. Sa contribution principale est expérimentale : rendre explicites les données retenues, les contrôles de recouvrement, la portée du verdict et les compromis observés entre architectures.

Les publications emploient des jeux de données, des unités d’analyse et des politiques d’annotation différents. Un taux élevé obtenu sur des fonctions ou sur du bytecode n’est pas directement comparable à un F1 macro calculé sur des contrats sources. Le tableau 1.3 expose donc les principes des travaux connexes, sans établir un classement à partir de scores hétérogènes.

La qualité de la séparation des données mérite une attention particulière. Deux contrats peuvent partager une structure ou une provenance tout en différant par quelques noms. Une partition aléatoire naïve peut ainsi placer des variantes très proches dans l’entraînement et dans le test. Le regroupement par familles et les empreintes contrôlées réduisent ce risque, sans garantir la détection de tous les clones approximatifs.

Trois choix définissent le positionnement de SMART BUG : conserver le contexte complet, séparer calibration et validation, et préparer une comparaison commune du CNN–BiLSTM actif avec XGBoost et CodeBERT. La revue de littérature présente les démarches existantes. Le classement expérimental reposera exclusivement sur les prédictions mesurées des trois modèles sur les mêmes contrats.

Une supériorité générale de l’architecture hybride ne peut pas être supposée : les différences entre méthodes peuvent être faibles et les annotations imparfaites. Le chapitre suivant décrit les mécanismes retenus pour interpréter les résultats et les reproduire. L’étude mesure la classification globale des contrats et précise les erreurs de chaque méthode.

Chapitre 2

Architecture et modélisation

La conception de SMART BUG organise le passage d'un corpus annoté à une application utilisable pour l'examen de contrats. L'architecture sépare la préparation des données, la construction du réseau, l'évaluation et l'inférence. Cette séparation facilite la traçabilité : un résultat doit pouvoir être rattaché aux données, au vocabulaire, aux poids et au seuil qui l'ont produit.

Le système comprend une chaîne hors ligne et une chaîne d'utilisation. La première construit les partitions et entraîne les modèles. La seconde reçoit un code Solidity, applique le même prétraitement puis restitue une probabilité calibrée et un verdict. Un moteur statique apporte des alertes complémentaires ; une base locale conserve l'historique des analyses.

Ce chapitre détaille les flux, les règles de constitution du corpus et la représentation hiérarchique du contrat. Les diagrammes UML présentent les responsabilités et les interactions à un niveau adapté à l'implémentation. Ils servent à expliquer le logiciel existant, sans ajouter de services ou de fonctions qui n'ont pas été réalisés.

2.1 Architecture du système SMART BUG

La figure 2.1 présente les principaux éléments du système. Les données sources sont normalisées et regroupées avant la séparation. Le vocabulaire est appris exclusivement sur l'entraînement. Après apprentissage et validation, la calibration fixe la température et le seuil. Le modèle retenu est décrit par un manifeste utilisé lors de l'inférence.

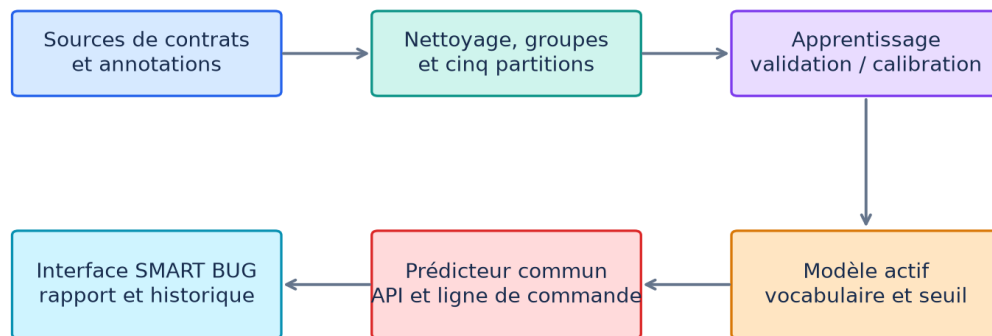


Figure 2.1 – Chaîne de données, apprentissage et utilisation.
Source : élaboration personnelle d'après le système implémenté.

L'application web repose sur FastAPI et une interface en HTML, CSS et JavaScript. Le fichier Solidity est transmis au service d'analyse. Celui-ci appelle le prédicteur commun à l'application et à la ligne de commande, puis les modules d'analyse statique et de performance. Les résultats sont affichés avec leurs origines respectives et enregistrés dans SQLite.

Le chargement des ressources vérifie les empreintes du fichier de poids et du vocabulaire. Le manifeste contrôle également la version du prétraitement et les paramètres de calibration. Cette vérification empêche de présenter comme évalué un assemblage incohérent de ressources. Le système ne réentraîne pas le réseau à chaque analyse et n'écrit pas les résultats sur une blockchain.

2.1.1 Construction et séparation du corpus

Les sources initiales totalisent 26 134 exemples. Le traitement place 2 440 exemples en quarantaine : 796 portant des labels contradictoires, 503 négatifs CGT insuffisamment couverts et 1 141 fonctions privées du contrat parent. La déduplication retire ensuite 5 456 doublons. Le corpus final contient 18 238 contrats ou représentations de contrats.

Tableau 2.1 – Composition du corpus final par partition.

Partition	Total	Label 0	Label 1	Groupes
Entraînement	11 959	6 072	5 887	9 196
Validation	1 709	867	842	1 310
Calibration	1 709	868	841	1 235
Test	1 709	867	842	1 295
Source réservée	1 152	6	1 146	1 054
Total	18 238	8 680	9 558	14 090

Source : configuration, données et résultats archivés de SMART BUG.

Les groupes réunissent les variantes liées par les critères de famille, de structure ou de provenance. Tous les groupes contenant CGT sont réservés à l'évaluation hors source. Le reste est distribué par une stratification groupée en dix plis, avec la graine 42 : le pli 0 pour le test, le pli 1 pour la validation, le pli 2 pour la calibration et les autres pour l'entraînement.

Les groupes traversant plusieurs provenances restent indivisibles. Les sources historiques comprennent notamment DAppSCAN, ScrawlID, des contrats audités avec Slither et des collections liées à SmartBugs ou Zeus. Leurs labels sont hérités de leur collecte ; nous ne les présentons pas tous comme vérifiés manuellement. CGT signifie Consolidated Ground Truth (SALZER et al., s. d.).

2.1.2 Prétraitement et prévention des fuites

Le prétraitement commence par une analyse lexicale qui retire les commentaires sans confondre ceux-ci avec le contenu des chaînes. Cette étape évite qu’une annotation présente en commentaire révèle directement le label. Les identifiants utilisateur sont renommés de manière cohérente à l’intérieur du contrat ; les chaînes et les longues adresses sont remplacées par des catégories.

La normalisation conserve les autres nombres pour l’entrée du réseau. Une représentation structurelle plus agressive sert au regroupement, mais n’est pas utilisée comme entrée à sa place. Cette distinction permet de rapprocher certaines variantes tout en conservant des informations lexicales utiles à la classification.

Le vocabulaire possède 8 000 entrées et provient uniquement de l’entraînement. Les valeurs inconnues sont représentées par leurs octets UTF-8 encadrés par un séparateur réservé, plutôt que toutes fusionnées dans un seul identifiant. Ce mécanisme préserve le contenu du token inconnu après normalisation. Les octets et marqueurs réservés font partie du vocabulaire.

Les séquences encodées sont découpées en fenêtres de 256 positions. Toutes les fenêtres sont conservées, y compris la dernière. Le remplissage nécessaire aux lots est masqué lors des agrégations. Un contrat porte une seule cible ; attribuer automatiquement son label positif à chacune de ses fenêtres introduirait une supervision incorrecte lorsque la faiblesse ne concerne qu’une partie du code.

L’audit indépendant recalcule les représentations et vérifie les labels, les groupes et six clés de recouvrement entre partitions. Il a contrôlé les 18 238 enregistrements sans différence d’encodage ni recouvrement selon ces critères. Cette observation ne prouve pas l’absence de fragments de bibliothèques communs ou de clones sémantiquement proches. Les empreintes garantissent une propriété précise sur les représentations comparées, et non une indépendance sémantique absolue.

2.1.3 Réseau hiérarchique retenu

Le réseau actif comporte 277 017 paramètres entraînaibles. Chaque fenêtre passe par un embedding de dimension 32 puis par 24 filtres convolutifs de taille 5, avec activation ReLU. Les moyennes et maxima masqués produisent un vecteur de 48 dimensions par fenêtre.

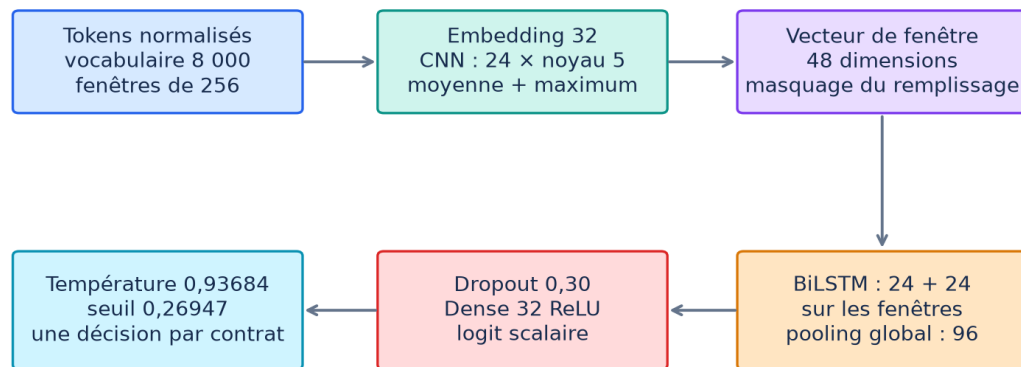


Figure 2.2 – Architecture du modèle actif CNN-BiLSTM.
Source : élaboration personnelle d’après le système implémenté.

Le BiLSTM possède 24 unités par direction et traite la séquence de ces vecteurs. Une seconde agrégation par moyenne et maximum produit 96 caractéristiques. Un dropout de 0,30 précède une couche dense de 32 unités ReLU et une sortie scalaire. Cette sortie est le logit utilisé par la perte binaire, puis par la calibration.

Les poids sont appris depuis zéro : aucun embedding ni encodeur pré-entraîné n’intervient dans le CNN-BiLSTM actif. L’ordre des fenêtres est disponible pour la récurrence, mais les agrégations compressent leur contenu. Le modèle estime une propriété globale du contrat et ne fournit pas directement une localisation. Son protocole de comparaison avec XG-Boost et CodeBERT est présenté au troisième chapitre.

2.2 Modélisation en UML

2.2.1 Diagramme de classes

Le diagramme de la figure 2.3 représente les objets manipulés et leurs responsabilités. Contrat et Analyse sont des entités conceptuelles ; le code utilise aussi des dictionnaires et des fonctions. SmartContractPredictor correspond à une classe réellement implémentée. Le stockage est représenté comme un module, car il repose sur des fonctions SQLite.

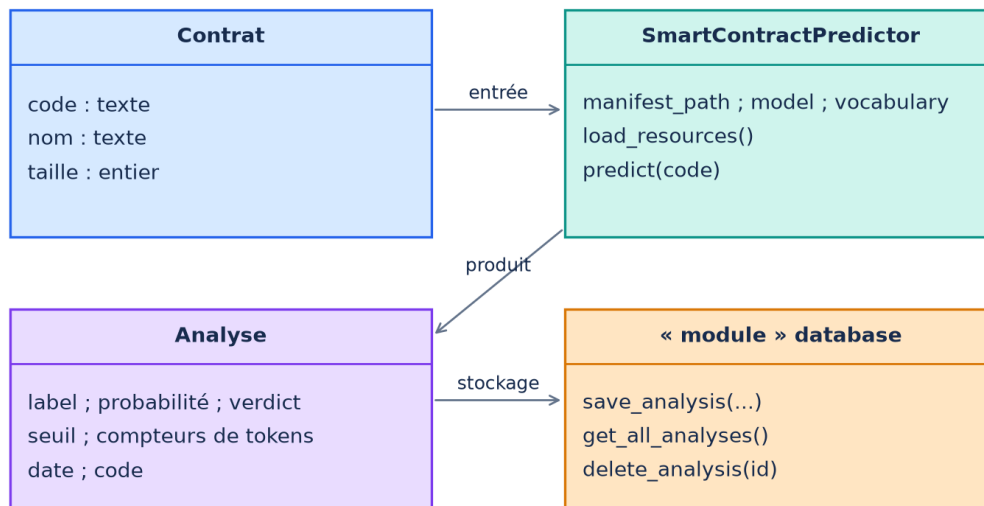


Figure 2.3 – Modèle conceptuel des données et du prédicteur.
Source : élaboration personnelle d’après le système implémenté.

Le prédicteur charge les ressources une seule fois, vérifie leur cohérence et produit la décision. L’analyse conserve notamment le nom du fichier, les probabilités, le verdict, des compteurs de tokens, le code et la date. Le module de base de données assure l’enregistrement, la consultation et la suppression.

Cette modélisation évite de confondre une dépendance logicielle avec une nouvelle entité métier. Le manifeste et le vocabulaire sont des ressources du prédicteur. Les annotations du corpus appartiennent à la phase expérimentale ; elles ne sont pas demandées à l’utilisateur lorsqu’il soumet un contrat à l’application.

2.2.2 Diagramme de cas d'utilisation

L'utilisateur soumet un fichier Solidity, consulte les résultats puis peut retrouver les analyses précédentes. L'interface donne également accès aux informations du modèle et du corpus. Ces fonctions correspondent aux vues existantes et aux routes de l'application ; le diagramme ne suppose pas de gestion de comptes ou de rôles qui ne serait pas implémentée.

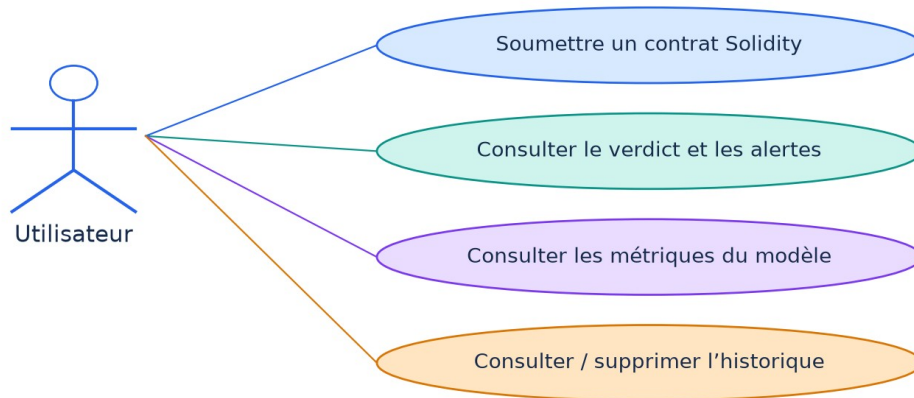


Figure 2.4 – Cas d'utilisation de l'application SMART BUG.
Source : élaboration personnelle d'après le système implémenté.

Le rapport distingue le verdict neuronal des alertes statiques. La probabilité est associée au modèle actif et au seuil enregistré, tandis que les alertes identifient des motifs à examiner. Le choix d'une présentation séparée facilite la lecture des désaccords possibles entre les deux méthodes.

L'historique permet de consulter le détail d'une analyse ou de supprimer un enregistrement. Il ne constitue pas un corpus d'apprentissage automatiquement réinjecté dans le réseau. Une telle réutilisation exigerait un processus d'annotation et de validation spécifique, faute de quoi les prédictions du système pourraient devenir leurs propres labels.

2.2.3 Diagramme de séquence

La séquence simplifiée décrit une analyse réussie. Le serveur valide le fichier, obtient le score du prédicteur et les alertes statiques, puis prépare l'enregistrement et la réponse. Une erreur interrompt ce parcours et donne lieu à un message explicite.

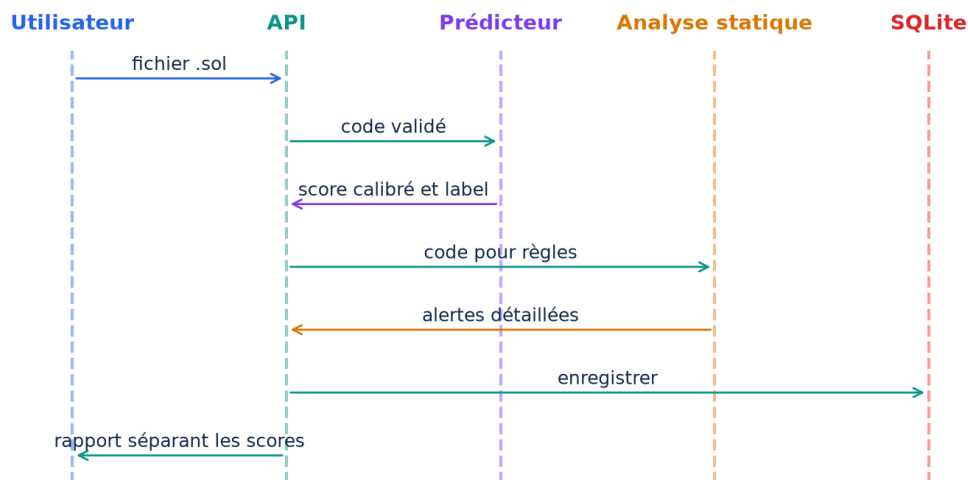


Figure 2.5 – Interactions principales lors d’une analyse.
Source : élaboration personnelle d’après le système implémenté.

2.2.4 Diagramme d’états-transitions

Les états de la figure 2.6 résument le parcours visible. Le retour à l’attente permet de soumettre un autre contrat. Une erreur peut provenir d’une entrée refusée ou d’une ressource du modèle indisponible ; elle ne doit pas être remplacée par un verdict de sécurité.

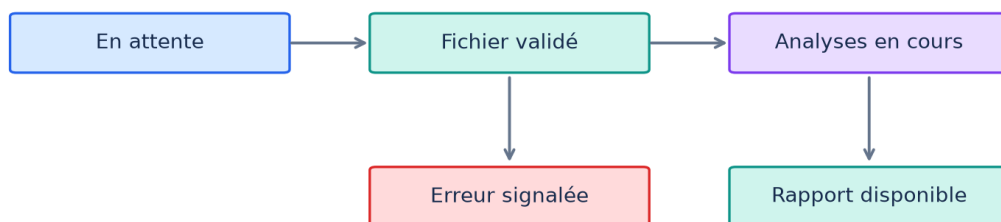


Figure 2.6 – États simplifiés du traitement d’un contrat.
Source : élaboration personnelle d’après le système implémenté.

2.2.5 Organisation des fichiers et reproductibilité

Le tableau 2.2 relie les opérations aux principaux fichiers. Il précise notamment le script qui orchestre l'entraînement, la validation, la calibration et le test du modèle actif. Cette organisation permet de remonter d'une métrique publiée à son calcul et aux ressources utilisées.

Tableau 2.2 – Principaux fichiers et responsabilités.

Fichier ou dossier	Rôle
src/build_dataset.py	Nettoyage, groupes et partitions
src/audit_dataset.py	Audit indépendant des représentations
src/preprocessing.py	Tokenisation et encodage communs
src/model.py	Construction du CNN-BiLSTM actif
src/train.py	Entraînement, validation, calibration et test
src/experiment.py	Lots et calcul des métriques
config/model.json	Hyperparamètres du modèle actif
src/predictor.py	Inférence commune au web et à la commande
src/run_model_benchmark.py	Comparaison CNN-BiLSTM / XGBoost / CodeBERT
config/benchmark_models.json	Réglages des concurrents et protocole

Source : configuration, données et résultats archivés de SMART BUG.

Les résultats actifs sont archivés dans `results/training` ; la comparaison `cnn-xgboost-codebert-20261005-final`, dans `results/benchmark`. Les historiques, prédictions individuelles et manifestes permettent de recalculer les indicateurs. Les poids actifs sont désignés par `models/active_model.json` ; le chargement vérifie leur empreinte SHA-256 ainsi que celle du vocabulaire.

La reproductibilité repose sur les configurations, les graines 42, 73 et 101, les partitions conservées et les versions de l'environnement enregistrées. Des bibliothèques ou matériels différents peuvent modifier certains calculs. Le commit 2df2580 documente l'expérience du modèle actif. La préparation comparative d'octobre 2026 possède sa configuration et ses manifestes distincts ; elle ne modifie pas les preuves historiques.

Chapitre 3

Investigation expérimentale

3.1 Description de l'application

SMART BUG est une application locale d'aide à l'analyse de sécurité des smart contracts Solidity. L'utilisateur fournit un fichier .sol ; l'application affiche une probabilité calibrée, un verdict global et des alertes issues de règles statiques. Les informations sur le modèle, le jeu de données et l'historique complètent le rapport.

L'expérimentation conserve le CNN-BiLSTM actif sous TensorFlow, dont le checkpoint a été sélectionné sur validation. Elle ajoute XGBoost, appris depuis zéro sur TF-IDF et évalué sur les mêmes partitions. CodeBERT a été envisagé comme troisième modèle, mais son entraînement complet n'a pas été réalisé. Les résultats publiés proviennent uniquement des calculs achevés.

Les observations du modèle actif proviennent des expériences archivées de septembre 2026 ; XGBoost a été entraîné et évalué en octobre 2026. Les métriques reprennent les fichiers sauvegardés. Les trois schémas comparatifs décrivent le protocole commun et le fine-tuning prévu de CodeBERT ; ils ne représentent aucune mesure simulée.

3.1.1 Technologies utilisées et hyperparamètres

Python porte la préparation des données. TensorFlow/Keras implémente le CNN-BiLSTM actif ; XGBoost entraîne les arbres et PyTorch adapte CodeBERT. Scikit-learn fournit TF-IDF et des métriques. FastAPI expose les services ; HTML, CSS et JavaScript constituent l'interface, et SQLite conserve les analyses. Le protocole documente le matériel effectivement utilisé pour chaque calcul.

Tableau 3.1 – Réglages du CNN-BiLSTM actif.

Paramètre	Valeur
Graines initiales / retenue	42, 73, 101 / 73
Embedding / filtres	32 / 24
Noyau / unités BiLSTM	5 / 24 par sens
Dropout / dense	0,30 / 32
Optimiseur / taux	Adam / 0,001
Époques max. / patience	12 / 3
Lot de référence	32, réduit selon longueur
Threads CPU	4

Source : configuration, données et résultats archivés de SMART BUG.

XGBoost utilise au plus 600 arbres, profondeur 6, taux 0,05, sous-échantillonnages des lignes et colonnes à 0,80, poids minimal d'enfant 2 et régularisation L2 de 1. La méthode hist et une patience de 40 itérations sur validation ont été appliquées. La graine 101 est retenue selon la log-loss de validation ; la prédiction utilise ses 598 premières itérations, sélectionnées sans consulter le test.

TF-IDF emploie unigrammes et bigrammes, au plus 20 000 caractéristiques, une fréquence minimale de deux documents et une fréquence de terme sous-linéaire. Son vocabulaire est appris sur l'entraînement uniquement. XGBoost a été entraîné avec les graines 42, 73 et 101 ; CodeBERT suit les mêmes graines prévues. Le CNN comparé reste le checkpoint actif de graine 73, sélectionné auparavant sur validation.

3.1.2 Mise en œuvre de l'entraînement

Le script `src/train.py` construit le modèle et parcourt les contrats par lots adaptés à leur longueur. La perte binaire est calculée à partir des logits. Adam ajuste les poids avec un taux initial de 0,001. L'arrêt anticipé surveille la perte de validation, avec une patience de trois époques ; les meilleurs poids sont rechargés avant l'évaluation.

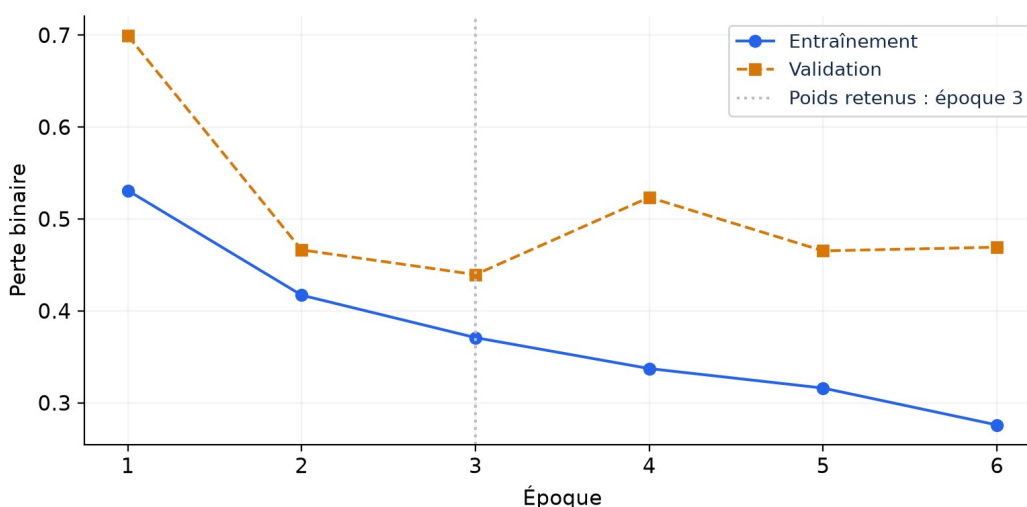


Figure 3.1 – Pertes d'apprentissage et de validation, graine 73.

Source : résultats expérimentaux archivés de SMART BUG.

Les graines 42, 73 et 101 ont exécuté respectivement six, six et huit époques. La graine 73 est sélectionnée selon la log-loss de validation ; son meilleur état correspond à la troisième époque. La figure 3.1 montre que la perte d'entraînement continue à baisser après ce point, alors que celle de validation augmente. Poursuivre l'optimisation ne garantit donc pas une meilleure généralisation.

La moyenne du F1 macro de validation des trois graines atteint 79,36 %, avec un écart-type de 1,39 point. Le choix final ne repose pas sur le test. Les historiques conservés rendent visible le critère de sélection et évitent de confondre la dernière époque exécutée avec l'époque dont les poids sont utilisés.

3.1.3 Intégration et cohérence de l'inférence

La courbe d'exactitude complète la lecture de la perte. L'exactitude de validation est plus élevée à la sixième époque qu'à la troisième, mais le critère prédéfini reste la log-loss. Modifier ce critère après observation des courbes reviendrait à changer la règle de sélection.

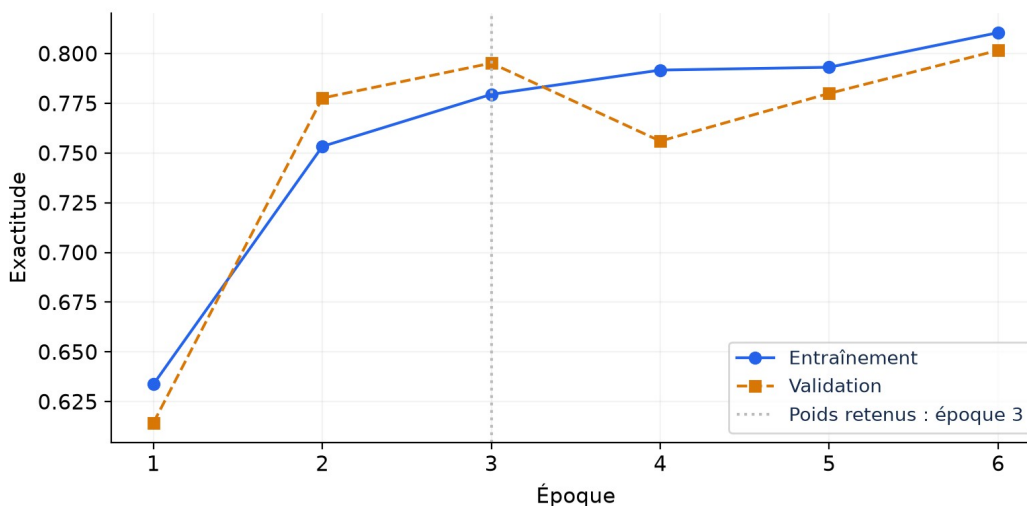


Figure 3.2 – Exactitudes d'apprentissage et de validation, graine 73.
Source : résultats expérimentaux archivés de SMART BUG.

L'application et la commande de prédiction utilisent `src/predictor.py`. Elles appliquent le même nettoyage, le même vocabulaire et le même découpage. La température retenue vaut 0,9368407 et le seuil 0,2694744. Un score inférieur à 0,5 peut donc conduire à une alerte positive si le seuil actif est dépassé ; ce comportement traduit l'objectif de rappel, et non une incohérence de l'interface.

Les contrôles logiciels couvrent notamment le traitement des tokens finaux, le masquage, le rechargement du modèle et la concordance d'une prédiction avec les résultats archivés. La suite du projet comptait 32 tests réussis lors de la vérification finale du code. Ce contrôle logiciel est distinct de la validité des labels et de la généralisation à de nouveaux projets.

3.2 Résultats

3.2.1 Performances du modèle actif

Le test comporte 1 709 exemples, dont 842 positifs et 867 négatifs. Après calibration, SMART BUG obtient une exactitude de 82,15 %, un F1 macro de 82,10 % et un rappel positif de 88,95 %. La matrice de confusion explicite les décisions correspondant à ces scores.

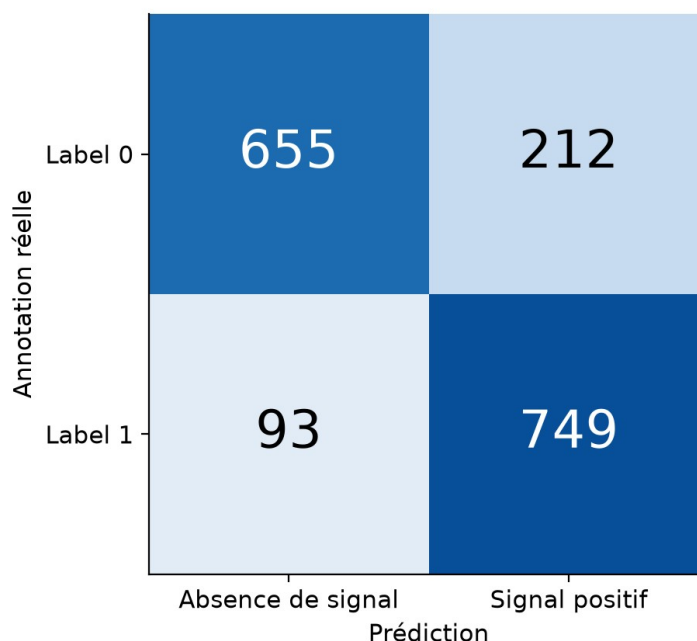


Figure 3.3 – Matrice de confusion du modèle actif calibré.
Source : résultats expérimentaux archivés de SMART BUG.

Le réseau identifie 749 positifs et manque 93 exemples annotés comme vulnérables. Il classe correctement 655 négatifs, mais produit 212 faux positifs. Le taux de faux positifs atteint ainsi 24,45 % des négatifs, et la précision positive 77,94 %. Une utilisation pratique doit tenir compte du travail de vérification associé à ces alertes.

Avec les sorties brutes et le seuil 0,5, le réseau produisait 176 faux négatifs et 124 faux positifs. L'ajustement du point de décision réduit donc les omissions tout en augmentant les alertes inutiles. Le rappel mesuré reste inférieur à la cible de 90 % fixée sur la calibration, ce qui illustre la variation entre partitions.

3.2.2 Probabilités et calibration

Tableau 3.2 – Résultats du test pour le CNN-BiLSTM actif.

Méthode	Exact. (%)	F1 macro (%)	Rappel (%)
CNN-BiLSTM brut	82,45	82,41	79,10
CNN-BiLSTM calibré	82,15	82,10	88,95

Source : configuration, données et résultats archivés de SMART BUG.

La température ne modifie pas le classement des scores : la ROC AUC du réseau reste proche de 0,907. Le score de Brier passe de 0,125856 à 0,125753 et l'ECE de 5,65 % à 5,12 %. L'amélioration probabiliste observée est modeste. La baisse du seuil explique surtout le changement du rappel et du nombre d'alertes.

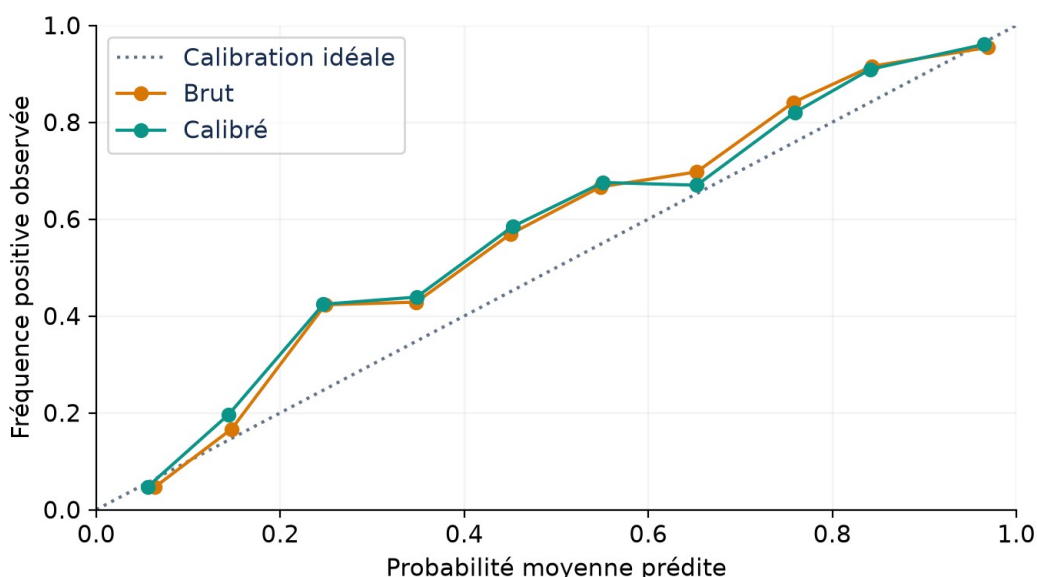


Figure 3.4 – Fiabilité des probabilités sur le test, dix intervalles.

Source : résultats expérimentaux archivés de SMART BUG.

Le diagramme confronte la probabilité moyenne prédite à la fréquence positive observée dans chaque intervalle. Une courbe proche de la diagonale indique une meilleure correspondance agrégée. L'estimation reste dépendante du nombre d'exemples par intervalle ; elle ne garantit pas la justesse d'une probabilité individuelle. Les performances hors source, discutées ensuite, montrent aussi les limites du transfert de cette calibration.

3.2.3 Comparaison du CNN-BiLSTM, de XGBoost et de CodeBERT

Le tableau présente les résultats du CNN-BiLSTM actif et du checkpoint XGBoost sélectionné sur validation. CodeBERT est un candidat non entraîné ; les cases à compléter indiquent une absence de résultat. Chaque ligne mesurée représente un checkpoint choisi avant l'évaluation du test, avec son seuil ajusté sur calibration.

Tableau 3.3 – État de la comparaison à trois modèles.

Méthode	F1 macro (%)	Rappel (%)	FPR (%)
CNN-BiLSTM actif	82,10	88,95	24,45
XGBoost / TF-IDF	89,41	89,43	10,61
CodeBERT (candidat)	Calcul à compléter	Calcul à compléter	Calcul à compléter

Source : configuration, données et résultats archivés de SMART BUG.

CNN et XGBoost : checkpoints sélectionnés sur validation. CodeBERT : candidat non entraîné.

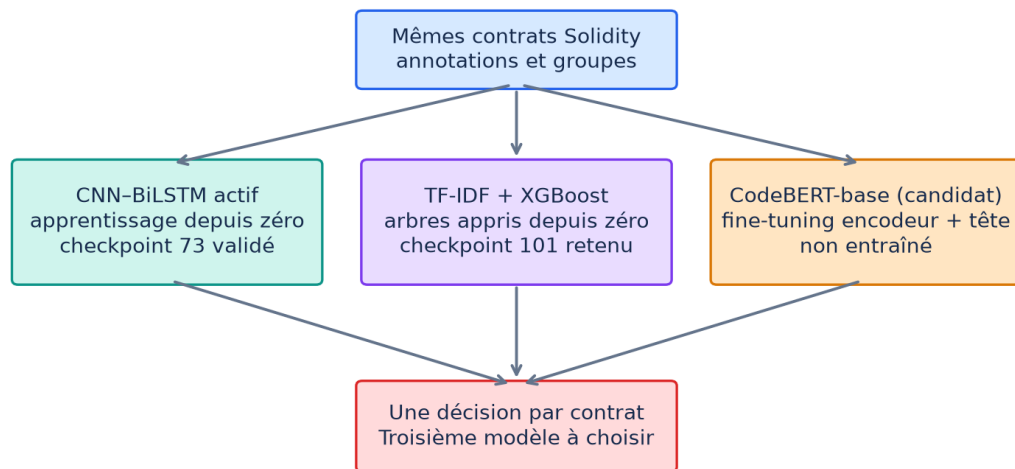


Figure 3.5 – Deux modèles évalués et un candidat supplémentaire.

Source : protocole comparatif ; évaluation CodeBERT à compléter.

Les modèles classent chacun le contrat complet. XGBoost exploite TF-IDF ; CodeBERT emploiera toutes les fenêtres. La log-loss de validation de XGBoost vaut 42 : 0,3244 ; 73 : 0,3271 ; 101 : 0,3197. Cette variabilité concerne la validation avant calibration ; le tableau n'est pas une moyenne des trois graines sur le test.

3.2.4 Protocole commun d'apprentissage et d'évaluation

Les cinq partitions et leurs identifiants restent identiques : 11 959 exemples d'entraînement, puis 1 709 pour chacun des ensembles de validation, calibration et test. La source réservée contient 1 152 exemples. Aucun modèle ne doit recevoir les annotations du test pour ajuster ses poids ou ses seuils.

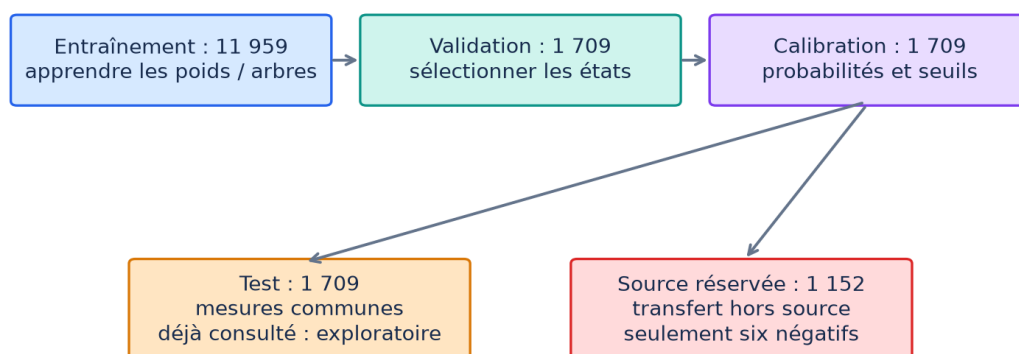


Figure 3.6 – Rôle des cinq partitions dans la comparaison.
Source : protocole comparatif ; évaluation CodeBERT à compléter.

La validation choisit les arbres ou le checkpoint CodeBERT. La calibration ajuste la température et les seuils visant 90 % de rappel ou 10 % de faux positifs. Les taux effectivement observés sur test doivent aussi être rapportés. Un objectif fixé sur calibration n'est pas une garantie sur de nouveaux contrats.

Le protocole est décrit dans `reports/protocole_cnn_xgboost_codebert.md` et `config/benchmark_models.json`. Le script `src/run_model_benchmark.py` pilote les calculs. Leur existence ne prouve pas que les entraînements soient terminés. Les résultats et les incertitudes ne seront ajoutés qu'après vérification des sorties.

3.2.5 Fine-tuning, calibration et coût de calcul

Le protocole CodeBERT-base prévoit AdamW, taux 0,00002, décroissance des poids 0,01, dropout 0 et au plus trois époques. Les fenêtres ont 256 sous-tokens, marqueurs compris. Une tête linéaire lit les états CLS ; la moyenne des logits de toutes les fenêtres produit un seul logit et une seule perte par contrat.

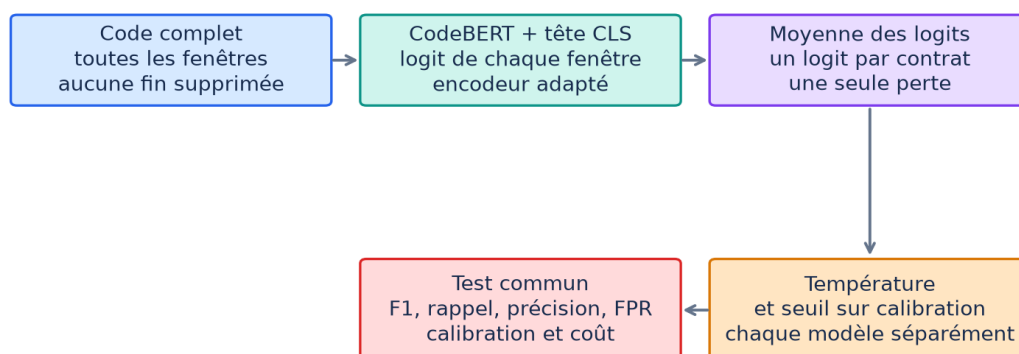


Figure 3.7 – Agrégation par contrat et calibration des décisions.

Source : protocole comparatif ; évaluation CodeBERT à compléter.

Le lot effectif prévu contient huit contrats ; quatre fenêtres sont traitées à la fois pour maîtriser la mémoire. L'encodeur et la tête reçoivent les gradients. Les trois graines serviront à mesurer la variabilité du fine-tuning. Le découpage conserve la fin du contrat sans attribuer automatiquement son label à chaque fenêtre.

Le pilote CPU de CodeBERT extrapole environ 5,2 jours par graine, soit 15,5 jours pour trois graines, hors entrées-sorties et évaluation. Ce candidat n'a pas été entraîné complètement et n'a aucun score. Ce coût motive la recherche d'une alternative, encore non choisie. Le rapport final devra aussi comparer calibration, latence et coût sur les mêmes contrats. Aucune comparaison des vitesses n'est encore établie.

3.3 Discussion

Le CNN-BiLSTM actif obtient 82,10 % de F1 macro, 88,95 % de rappel et 24,45 % de faux positifs. XGBoost atteint respectivement 89,41 %, 89,43 % et 10,61 % sur les mêmes contrats. Le gain de F1 est de 7,31 points ; un bootstrap par groupes (1 000 réplifications) donne un IC à 95 % de [4,94 ; 10,80] points, conditionné aux checkpoints choisis. Ce test déjà observé reste exploratoire ; CodeBERT n'est pas évalué, et les scores XGBoost ne sont pas une moyenne des graines sur test.

Le transfert à la source réservée constitue une limite majeure. Pour le modèle actif, le rappel positif y atteint 89,01 %, mais le F1 macro n'est que de 47,73 % et l'ECE de 29,84 %. Les 1 152 exemples comprennent 1 146 positifs et seulement six négatifs. L'exactitude de 88,63 % est inférieure à celle d'une règle prédisant toujours positif, soit 99,48 %. Le taux de faux positifs ne peut pas être estimé de manière stable sur six négatifs.

La validité interne reste limitée par les labels hérités, les variantes de contrats et les indices de provenance. La mise en quarantaine et l'audit réduisent des incohérences connues, mais n'établissent pas une vérité terrain exhaustive. Un modèle peut apprendre des régularités propres à une source plutôt que les seuls mécanismes des vulnérabilités.

La validité externe demande un corpus indépendant représentatif des contrats futurs. Le test actuel a déjà été observé ; les comparaisons à venir seront donc exploratoires. La normalisation des identifiants peut affecter le transfert de CodeBERT. Des représentations et des coûts différents sont autorisés, mais doivent être décrits : une différence de score ne mesurerait pas uniquement l'effet du préentraînement.

Enfin, la validité du construit dépend du sens du verdict. Le réseau mesure une association à des annotations binaires, tandis que l'audit de sécurité vise des comportements exploitables et leur contexte. SMART BUG est utile pour prioriser l'examen de contrats ; ses probabilités et ses alertes doivent être examinées conjointement par un évaluateur compétent.

3.4 Potentielles améliorations

La première priorité concerne les données. Il faut réexaminer les cas placés en quarantaine, conserver les désaccords d’annotation et enrichir les sources avec des négatifs vérifiés. L’évaluation de l’absence d’une faiblesse doit préciser quelles propriétés ont été contrôlées. Une annotation experte, accompagnée de scénarios ou de justifications, serait plus informative qu’un simple label global.

Un nouveau test devrait être constitué avant tout ajustement supplémentaire, avec séparation par projets, familles et, si possible, période de publication. Les critères de sélection et les seuils seraient fixés sans le consulter. Un ensemble hors source comprenant suffisamment de contrats de chaque classe permettrait de mieux estimer les faux positifs et la stabilité de la calibration.

Sur le plan expérimental, la priorité est de choisir et d’entraîner un troisième modèle dans les ressources disponibles, puis d’évaluer le checkpoint retenu sur validation avec le même protocole. CodeBERT reste un candidat non entraîné. Les critères, les seuils de calibration, le coût et la variabilité doivent rester documentés. Une étude ultérieure pourrait examiner les graphes ou réentraîner les modèles dans une nouvelle campagne commune, accompagnée d’un test indépendant.

La localisation des vulnérabilités constitue une autre extension. Elle demande des labels au niveau des lignes ou des fonctions et une évaluation propre de la localisation. Une carte d’attention ou une activation élevée ne doit pas être assimilée sans vérification à une explication causale du défaut. L’interface pourrait afficher plus explicitement les désaccords entre règles et réseau.

Enfin, une étude d’usage permettrait de mesurer le temps d’audit économisé, la proportion d’alertes confirmées et les cas manqués. La surveillance de la distribution des entrées et la réévaluation périodique de la calibration accompagneraient une utilisation prolongée. L’amélioration prioritaire du travail passe ainsi par des preuves plus solides et des objectifs d’usage mesurables, autant que par de nouveaux modèles.

Conclusion

Ce mémoire a étudié une approche d'apprentissage profond pour la détection automatisée des vulnérabilités des smart contracts Solidity. SMART BUG associe un CNN par fenêtres et un BiLSTM hiérarchique afin de traiter le code complet. Le corpus a été nettoyé, regroupé et réparti entre cinq usages distincts. Le prétraitement commun, les manifestes et la calibration dédiée rendent la chaîne expérimentale traçable.

Le modèle actif est entraîné depuis zéro. Sur 1 709 exemples de test, il obtient un F1 macro de 82,10 % et un rappel positif de 88,95 %, avec 93 faux négatifs et 212 faux positifs. XGBoost, évalué sur le même test, atteint un F1 macro de 89,41 %. La comparaison principale confronte ces deux modèles ; CodeBERT a été étudié comme candidat non entraîné. Le troisième modèle définitif reste à choisir : le classement final et la généralisation à d'autres corpus ne sont pas établis.

La réponse à la problématique est donc nuancée : l'apprentissage profond peut aider à prioriser l'audit, à condition d'en préciser le périmètre et les erreurs. Les annotations imparfaites, le déséquilibre hors source et l'absence d'un nouveau test indépendant limitent les conclusions. Les perspectives prioritaires sont l'enrichissement des données vérifiées, une évaluation indépendante et l'étude de la localisation. Le système réalisé constitue une base expérimentale reproductible pour poursuivre ces travaux.

Bibliographie

Livres

GOODFELLOW, I., BENGIO, Y. et COURVILLE, A. (2016). Deep Learning. MIT Press. deeplearningbook.org

Articles et communications

FEIST, J., GRIECO, G. et GROCE, A. (2019). Slither: A Static Analysis Framework for Smart Contracts. WETSEB. DOI : 10.1109/WETSEB.2019.00008. arxiv.org/abs/1908.09878

FERREIRA, J. F., CRUZ, P., DURIEUX, T. et ABREU, R. (2020). SmartBugs: A Framework to Analyze Solidity Smart Contracts. Prépublication [arXiv:2007.04771](https://arxiv.org/abs/2007.04771). arxiv.org/abs/2007.04771

GUO, C., PLEISS, G., SUN, Y. et WEINBERGER, K. Q. (2017). On Calibration of Modern Neural Networks. PMLR, 70, 1321–1330. proceedings.mlr.press/v70/guo17a

KIM, Y. (2014). Convolutional Neural Networks for Sentence Classification. EMNLP, 1746–1751. DOI : 10.3115/v1/D14-1181. aclanthology.org/D14-1181

TANN, W. J.-W., HAN, X. J., SEN GUPTA, S. et ONG, Y.-S. (2018). Towards Safer Smart Contracts: A Sequence Learning Approach to Detecting Security Threats. Prépublication [arXiv:1811.06632](https://arxiv.org/abs/1811.06632), révisée en 2019. arxiv.org/abs/1811.06632

Articles et communications (suite)

FENG, Z. et al. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. Findings of EMNLP, 1536–1547. DOI : 10.18653/v1/2020.findings-emnlp.139. aclanthology.org/2020.findings-emnlp.139

CHEN, T. et GUESTRIN, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD, 785–794. DOI : 10.1145/2939672.2939785. arxiv.org/abs/1603.02754

ZHANG, L. et al. (2022). CBGRU: A Detection Method of Smart Contract Vulnerability Based on a Hybrid Model. Sensors, 22(9), 3577. DOI : 10.3390/s22093577. doi.org/10.3390/s22093577

ZHUANG, Y., LIU, Z., QIAN, P., LIU, Q., WANG, X. et HE, Q. (2020). Smart Contract Vulnerability Detection using Graph Neural Network. IJCAI, 3283–3290. DOI : 10.24963/ijcai.2020/454. ijcai.org/Proceedings/2020/454

Documentation et ressources du projet

SALZER, G. et contributeurs (s. d.). Consolidated Ground Truth (CGT) for Weaknesses of Ethereum Smart Contracts. Dépôt de données. github.com/gsalzer/cgt

SOLIDITY (s. d.). Security Considerations. Documentation officielle. docs.soliditylang.org — Security Considerations

KOMPANI KIPANGU, E. (2026). SMART BUG : code, configurations, corpus préparé et résultats expérimentaux. Dépôt du mémoire, commit 2df2580. GitHub — dépôt SMART BUG, état étudié

Références du protocole comparatif mises à jour le 5 octobre 2026. Les graphiques et tableaux de résultats proviennent des artefacts locaux cités au chapitre 2.